TU Dortmund

Fachprojekt Detektion von Sicherheitslücken

M.Sc. Johannes Düsing, M.Sc. Anemone Kampkötter

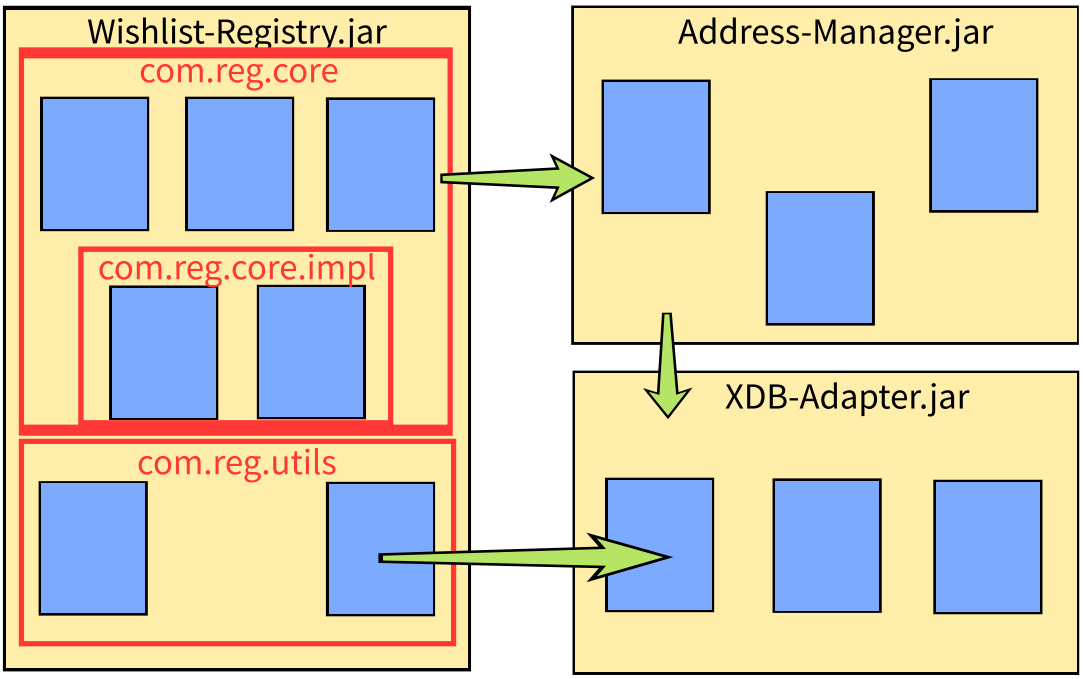
JProf. Dr.-Ing. Ben Hermann

Aufgabe 6: Architecture Validation

Ausgabe: 18.12.2024

Besprechung: 08.01.2025

Helfen Sie dem Weihnachtsmann, die Geschenke für Weihnachten zu retten! Santa hat schon lange den Überblick über sein Software-System zur Auswertung von Wunschzetteln verloren, und kurz vor dem heiligen Abend ist bei dem eigentlich einfachen Austausch der Sub-Komponente `XDB-Adapter` das System völlig zusammengebrochen. Dabei gab es eigentlich eine streng definierte Architektur für das System, nur die Klassen innerhalb von `Address-Manager.jar` und eine einzelne Klasse aus `Wishlist-Registry.jar` durften von Klassen innerhalb `XDB-Adapter` abhängen (siehe Abbildung).



Aber über die vielen Jahrhunderte der Wartung und Erweiterung wurde immer weiter von der eigentlichen Architektur abgewichen, sodass jetzt alle möglichen Klassen voneinander abhängen, und es zu unvorhergesehenen Fehlern kommt. Für diese Übung sollen Sie eine Analyse implementieren, welche eine vorgegebene Spezifikation der Software-Architektur einliest (*Soll-Zustand*) und überprüft, ob sich ein gegebenes System an diese Spezifikation hält, bzw. an welchen Stellen es gegen die Spezifikation verstößt.

6.1 Analyseaufgabe

Für den Kontext dieser Aufgabe betrachten wir drei Granularitätsebenen: JARs, Packages und Klassen. Es gilt:

- Eine Klasse A *nutzt* eine Klasse B, wenn A eine Methode von B aufruft (bzw. aufrufen könnte), oder auf ein Feld von B zugreift.

- Eine Package P *nutzt* eine Klasse B, wenn mindestens eine Klasse $A \in P$ die Klasse B nutzt.
- Ein Package P *nutzt* ein Package Q, wenn $\exists A \in P, B \in Q$ sodass A B nutzt.
- Analoge Definitionen für JARs.

Die Spezifikationsform für den Soll-Zustand ist fest vorgegeben, ein Beispiel dass sich auf die Abbildung oben bezieht findet sich in Listing 1. In Zeile 2 wird angegeben, dass grundsätzlich alle Nutzungen untersagt sind, die nicht explizit erlaubt wurden (Allow-Listing). Dies kann auf Wunsch invertiert werden, sodass alle Nutzungen grundsätzlich erlaubt sind, außer solchen, die explizit verboten wurden (Ban-Listing). In den Zeilen 4 bis 15 wird eine Regel definiert, die es allen Klassen des Packages `com.reg.core` erlauben, die Klassen des JARs `Address-Manager.jar` zu nutzen. Dabei gibt es eine Ausnahme für Klassen des Packages `com.reg.core.impl`, diese dürfen nicht auf das JAR zugreifen. In den Zeilen 15 bis 19 wird die Nutzung der Klasse `com.xdb.adapter.Bar` innerhalb der Klasse `com.reg.util.Foo` explizit erlaubt.

```
1  {
2    "defaultRule": "FORBIDDEN",
3    "rules": [
4      {
5        "from": "com.reg.core",
6        "to": "Address-Manager.jar",
7        "type": "ALLOWED",
8        "except":[
9          {
10           "from": "com.reg.core.impl",
11           "to": "Address-Manager.jar",
12           "type": "FORBIDDEN"
13         }
14       ]
15     },{
16       "from": "com.reg.utils.Foo",
17       "to": "com.xdb.adapter.Bar",
18       "type": "ALLOWED"
19     }
20   ]
21 }
```

Listing 1: Spezifikationsform für die Softwarearchitektur

Abbildung 1 zeigt eine UML-Definition des Eingabeformats. Hier wird deutlich, dass die Ausnahmen in einer `except`-Klausel beliebig tief geschachtelt sein können, es kann also zB. Ausnahmen von den Ausnahmen einer Verbots-Regel geben.

Ihre Analyse soll eine solche Spezifikation im JSON Format einlesen und zunächst auf semantische Fehler überprüfen. Semantische Fehler in der Spezifikation wären zB. JAR- / Package- / Klassen-Namen, die nicht im Projekt existieren, oder Ausnahmen, die semantisch nicht zur eigentlichen Regel passen. Sie können entscheiden, ob dies zum Abbruch der Analyse führen soll, oder lediglich als Warnung ausgegeben wird. Anschließend soll ihre Analyse alle Methodenaufrufe und Feldzugriffe ausgeben, die gegen die Spezifikation verstoßen. Wie diese ausgegeben werden, bleibt Ihnen überlassen.

6.2 Evaluationsaufgabe

Erstellen Sie in jeder Gruppe eine eigene Ground Truth. Stellen Sie dafür zunächst einen eigenen Benchmark zusammen, bestehend aus 3 JARs, in denen insgesamt 20 Klassen mit einer beliebigen Paketzuweisung enthalten sind. Implementieren Sie genügend interessante Beziehungen zwischen den Klassen, Packages und JARs. Um diese testen zu können, erstellen Sie außerdem eine Spezifikation nach der vorgegebenen Form wie in 1. Geben Sie schließlich für Ihre Ground Truth einen beliebigen Ausgabe-Bericht an, der die erwarteten Verstöße enthält.

Stellen Sie Ihre Ground Truth den beiden anderen Gruppen vor unserem nächstem Treffen zur Verfügung.

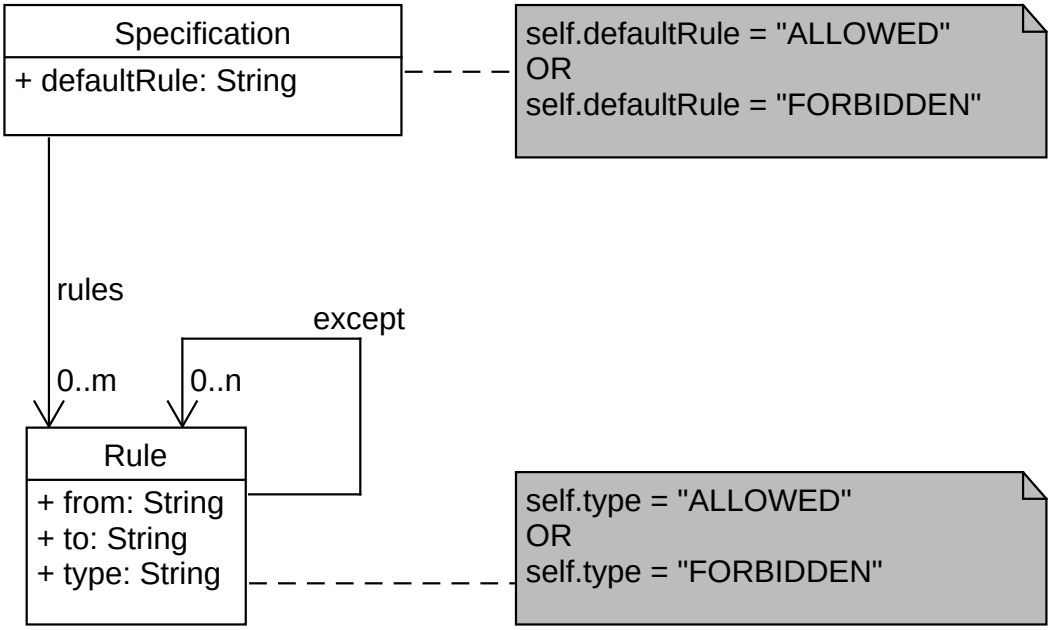


Abbildung 1: Formales Format der Spezifikation

Weitere Anforderungen:

- Stellen Sie sicher, dass Ihre Implementierung die geforderte Funktionalität korrekt umsetzt. Schreiben Sie mehrere Unit-Tests[1] für Ihren Codes und messen sie die Testabdeckung (Coverage). Stellen Sie dabei verschiedene Arten von Testabdeckung gegenüber.
- Stellen Sie uns Ihre Lösung wieder in Ihrem Git-Repository zur Verfügung.

[1] `https://www.baeldung.com/scala/scalatest`